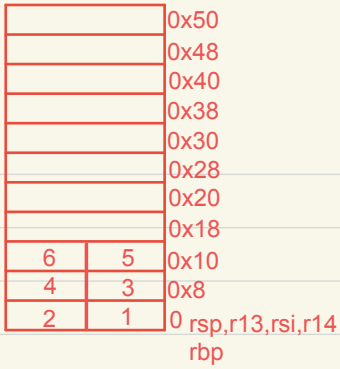




第一重循环当 r12d < 6
对每个数字进行迭代
r13: 地址计算
rbp: 指向数字

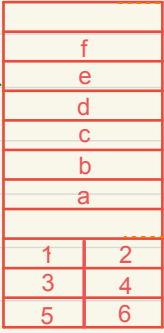
跳出循环, r12d==6

③ 同样迭代这6个数字
每次用7减去这个数字并
用结果替换掉这个数字

从内存地址(rdx+8)取8字节
放入 rdx直到 eax >= x
(eax 从 1 开始)

只要被迭代的数字x <= 1
就将8字节的0x6032d0写入
0x20(%rsp)开始的栈帧空间

只要被迭代的数字x > 1
通过循环 获得正确的
rdx并将其写入栈帧空间



数字x > 1

数字x <= 1

```
<+0>: push    %r14
<+2>: push    %r13
<+4>: push    %r12
<+6>: push    %rbp
<+7>: push    %rbx
<+8>: sub     $0x50, %rsp          分配80字节空间
<+12>: mov     %rsp, %r13
<+15>: mov     %rsp, %rsi
<+18>: call    0x40145c <read_six_numbers>
<+23>: mov     %rsp, %r14
<+26>: mov     $0x0, %r12d  作为迭代索引
<+32>: mov     %r13, %rbp
<+35>: mov     0x0(%r13), %eax 数字x
<+39>: sub     $0x1, %eax
<+42>: cmp     $0x5, %eax
<+45>: jbe     0x401128 <phase_6+52> 数字x减1完后<=5, 即数字x<=6
<+47>: call    0x40143a <explode_bomb>
<+52>: add     $0x1, %r12d
<+56>: cmp     $0x6, %r12d
<+60>: je      0x401153 <phase_6+95>
<+62>: mov     %r12d, %ebx
<+65>: movslq   %ebx, %rax
<+68>: mov     (%rsp, %rax, 4), %eax
<+71>: cmp     %eax, 0x0(%rbp)
<+74>: jne     0x401145 <phase_6+81>
<+76>: call    0x40143a <explode_bomb>
<+81>: add     $0x1, %ebx
<+84>: cmp     $0x5, %ebx
<+87>: jle     0x401135 <phase_6+65>
<+89>: add     $0x4, %r13
<+93>: jmp     0x401114 <phase_6+32>
<+95>: lea     0x18(%rsp), %rsi
<+100>: mov     %r14, %rax  rax=r14=rsp=栈顶
<+103>: mov     $0x7, %ecx
<+108>: mov     %ecx, %edx  edx=7
<+110>: sub     (%rax), %edx  减去数字x
<+112>: mov     %edx, (%rax)  数字x被替换
<+114>: add     $0x4, %rax  rax 指向下一个数字
<+118>: cmp     %rsi, %rax
<+121>: jne     0x401160 <phase_6+108>
<+123>: mov     $0x0, %esi  esi 清0, 直接跳到<+163>, 开始迭代
<+128>: jmp     0x401197 <phase_6+163>
<+130>: mov     0x8(%rdx), %rdx
<+134>: add     $0x1, %eax
<+137>: cmp     %ecx, %eax
<+139>: jne     0x401176 <phase_6+130>
<+141>: jmp     0x401188 <phase_6+148>
<+143>: mov     $0x6032d0, %edx
<+148>: mov     %rdx, 0x20(%rsp, %rsi, 2)
<+153>: add     $0x4, %rsi
<+157>: cmp     $0x18, %rsi
<+161>: je      0x4011ab <phase_6+183>
<+163>: mov     (%rsp, %rsi, 1), %ecx  ecx = 数字x
<+166>: cmp     $0x1, %ecx
<+169>: jle     0x401183 <phase_6+143>
<+171>: mov     $0x1, %eax
<+176>: mov     $0x6032d0, %edx
<+181>: jmp     0x401176 <phase_6+130>
<+183>: mov     0x20(%rsp), %rbx
<+188>: lea     0x28(%rsp), %rax
<+193>: lea     0x50(%rsp), %rsi
<+198>: mov     %rbx, %rcx
<+201>: mov     (%rax), %rdx  从栈帧里取值
<+204>: mov     %rdx, 0x8(%rcx)  并把值写到前一个值所代表的内存地址+8的内存里去
<+208>: add     $0x8, %rax
<+212>: cmp     %rsi, %rax
<+215>: je      0x4011d2 <phase_6+222>
<+217>: mov     %rdx, %rcx
<+220>: jmp     0x4011bd <phase_6+201>
<+222>: movq    $0x0, 0x8(%rdx)  栈帧里最后一个值所代表的内存地址+8的内存被写入0
<+230>: mov     $0x5, %ebp
<+235>: mov     0x8(%rbx), %rax  取出前面写入的值, 并当成内存地址,
<+239>: mov     (%rax), %eax  再从里面取值出来
<+241>: cmp     %eax, (%rbx)
<+243>: jge     0x4011ee <phase_6+250>
<+245>: call    0x40143a <explode_bomb>
<+250>: mov     0x8(%rbx), %rbx
<+254>: sub     $0x1, %ebp
<+257>: jne     0x4011df <phase_6+235>
<+259>: add     $0x50, %rsp
<+263>: pop     %rbx
```

以绿线箭头循环为线索划分
区域并跟踪循环逻辑以及何
时会跳出循环, 进行分析

说明数字x 是 unsigned 类型的

被迭代的数字必须 1<= x <=6 ①

索引+1后是否==6

索引不超过6, 就可以拿来获取栈帧中的6个数字
因为r12d已经先+1了, 所以最多只能索引后5个数字

从第一个数开始检查, 如果后续数字有跟
当前迭代数字一样的就爆炸 ②

使用rsi做迭代索引, 进行循环
往0x20(%rsp)-0x50(%rsp)
栈帧空间写入内存地址

第一个值
第二个值的地址
结束地址

栈帧里最后一个值所代表的内存地址+8的内存被写入0

取出前面写入的值, 并当成内存地址,
再从里面取值出来

⚠ 注意: 这里拿的是底32位来比较

$M[(\%rbx)] \geq M[M[0x8(\%rbx)]]$

3	0x6032f0	0x39c
4	0x603300	0x2b3
5	0x603310	0x1dd
6	0x603320	0x1bb
1	0x6032d0	0x14c
2	0x6032e0	0xa8

